

Máster en Inteligencia Artificial
para Liderazgo Técnico y Transformación Empresarial

Gestión de Tokens y Costes de Inferencia

Módulo 8 · Ingeniería de Modelos y LLMOps

Precios por modelo, prefix caching, routing, gestion del historial y break-even cloud vs GPU propio.

1. Introducción · 2. Qué es · 3. Cómo funciona · 4. Ejemplos reales
5. Errores comunes · 6. Aplicación práctica · 7. Relación con módulos · 8. Resumen ejecutivo

Guía de estudio detallada · +2.000 palabras

Gestión de Tokens y Costes de Inferencia

El coste de operar un sistema LLM en producción puede escalar de forma no intuitiva. Un prompt bien diseñado de 500 tokens que funciona perfectamente para 100 consultas al día cuesta \$5 al mes con GPT-4o. El mismo prompt procesando 1.000.000 de consultas al día cuesta \$50.000 al mes. La diferencia entre un sistema LLM económicamente viable y uno prohibitivo a escala frecuentemente está en la gestión cuidadosa de los tokens: cómo se construyen los prompts, qué información se incluye, cómo se gestiona el historial de conversación, y qué modelo se usa para cada tipo de tarea.

La gestión de tokens y costes no es solo una preocupación económica: es también una preocupación de rendimiento. Prompts más cortos producen menor latencia. Contextos más compactos reducen la tasa de distracción del modelo (el modelo puede "distrarse" con información irrelevante en prompts muy largos). Entender el perfil de coste de un sistema LLM es una competencia técnica fundamental para cualquier equipo que lleva IA a producción.

Cómo se factura la inferencia LLM

Tokens de entrada vs tokens de salida

Los proveedores de LLM cobran por separado los tokens de entrada (el prompt, incluyendo system prompt, historial, y contexto RAG) y los tokens de salida (la respuesta generada). Los tokens de salida son típicamente 3-5x más caros que los de entrada porque la generación autoregresiva es más costosa computacionalmente que el procesamiento del prompt. Para GPT-4o (abril 2025): \$2.50/M tokens de entrada, \$10/M tokens de salida. Para GPT-4o-mini: \$0.15/M entrada, \$0.60/M salida.

Prefix caching: ahorrar en prompts repetitivos

Los principales proveedores (OpenAI, Anthropic) ofrecen prefix caching: si el inicio del prompt (típicamente el system prompt) es idéntico entre consultas, el coste del prefijo cacheado se reduce al 50% o más. Para sistemas donde el system prompt tiene 2.000 tokens y se repite en cada consulta, el prefix caching puede ahorrar el 30-40% del coste total de tokens de entrada. Para que el caching sea efectivo, el system prompt debe ser fijo y colocado al principio del contexto.

Modelos pequeños vs grandes: la estrategia de routing

La diferencia de coste entre GPT-4o y GPT-4o-mini es de 15-17x por token. Para tareas simples donde GPT-4o-mini alcanza el mismo rendimiento que GPT-4o, el ahorro potencial es de 15-17x. El routing inteligente (clasificar cada consulta según dificultad y enviarla al modelo más económico que puede manejarla correctamente) es la estrategia de reducción de coste más impactante disponible.

Estrategias de optimización de tokens

Compresión de prompts

La compresión de prompts elimina tokens innecesarios sin degradar la calidad: eliminar espacios redundantes, puntuación innecesaria, y frases repetitivas (puede ahorrar 5-10% de tokens). Para contexto RAG: incluir solo los fragmentos más relevantes en lugar de recuperar $K=10$ cuando $K=3$ es suficiente. Para few-shot examples: usar el mínimo de ejemplos que produce el rendimiento requerido (evalúa la curva rendimiento/tokens). Para system prompts: revisar periódicamente y eliminar instrucciones redundantes o que el modelo cumple igualmente sin necesidad de instrucción explícita.

Gestión del historial de conversación

En sistemas conversacionales, el historial crece con cada turno. Sin gestión, después de 10-20 turnos el coste por consulta puede multiplicarse por 5-10x respecto al primer turno. Las estrategias de gestión del historial son: sliding window (mantener solo los últimos N turnos), resumen periódico (cada K turnos, comprimir el historial anterior en un resumen conciso), y extracción selectiva (mantener solo las partes del historial relevantes para el tema actual).

Streaming: reducir la latencia percibida sin reducir tokens

El streaming devuelve los tokens de la respuesta a medida que se generan, reduciendo el tiempo hasta el primer token visible (TTFT). Esto no reduce el número total de tokens ni el coste, pero mejora significativamente la experiencia de usuario: el usuario ve que el sistema está respondiendo inmediatamente en lugar de esperar que se complete toda la respuesta. Es especialmente importante para respuestas largas donde el tiempo de generación total puede ser de 10-30 segundos.

SECCIÓN 4 · EJEMPLOS REALES

- Reducción del 42% del coste mediante routing (empresa de e-commerce): un sistema de QA sobre productos usaba GPT-4o para todas las consultas. Análisis del tráfico mostró que el 70% eran preguntas simples (precio, disponibilidad, tallas). Implementaron routing: consultas simples → GPT-4o-mini, consultas complejas (comparaciones, recomendaciones personalizadas) → GPT-4o. Resultado: -42% coste total con calidad igual en los casos simples y superior en los complejos (que antes se perdían por ser el mismo modelo para todo).
- Gestión del historial en chatbot de soporte con sesiones largas: sin gestión, una sesión de 20 turnos consumía 8.000 tokens de historial. Con sliding window de 6 últimos turnos + resumen de los anteriores: 2.500 tokens de historial promedio. Ahorro del 68% en tokens de contexto para sesiones largas. La calidad no se degradó porque el resumen preservaba el contexto relevante.
- Prefix caching en asistente legal (bufete): el system prompt de 3.000 tokens (contexto del bufete, guías de estilo, instrucciones de compliance) es idéntico en todas las consultas. Con prefix caching de Anthropic, el 90% de las consultas reciben descuento del 90% en los tokens del system prompt. Ahorro mensual: ~\$2.400.

SECCIÓN 5 · ERRORES Y MALENTENDIDOS COMUNES

Error 1: No monitorizar el coste por consulta con granularidad. Sin métricas de tokens de entrada vs salida por tipo de consulta, es imposible identificar qué parte del sistema es responsable del mayor coste. Loggea `tokens_input`, `tokens_output`, y `coste_estimado` para cada llamada al LLM.

Error 2: Comprimir prompts hasta degradar la calidad. La compresión agresiva de prompts puede eliminar instrucciones que parecen redundantes pero que son necesarias para casos edge. Evalúa cualquier compresión de prompt sobre tu golden dataset para verificar que no hay degradación de calidad.

Error 3: No calcular el coste total de propiedad del modelo. El coste de tokens es el coste de operación. El coste total incluye también: tiempo de ingeniería para mantener los prompts, coste del sistema de monitorización, coste de evaluación continua (LLM-as-judge también tiene coste de tokens), y coste de las llamadas adicionales para self-consistency o reranking. El coste de los modelos de embedding, reranking y evaluación puede ser el 30-40% del coste total del sistema.

SECCIÓN 6 · APLICACIÓN PRÁCTICA

El proceso de análisis de coste de un sistema LLM: usa tiktoken (OpenAI) o el tokenizador del modelo para medir exactamente los tokens de cada componente del prompt (system prompt, historial, contexto RAG, consulta del usuario). Identifica los componentes más costosos. Estima el coste mensual con los volúmenes de uso esperados. Define objetivos de reducción de coste para los componentes más caros. Implementa las optimizaciones más impactantes primero (routing, prefix caching, gestión del historial).

Dashboard de coste recomendado para monitorización: `tokens_input_total_diario`, `tokens_output_total_diario`, `coste_total_diario`, `coste_por_consulta_medio`, `coste_por_consulta_p99`, `distribución_de_modelos_usados` (% consultas por modelo), alertas cuando `coste_diario > umbral`. Herramientas: Langfuse tiene dashboards de coste integrados; Helicone es un proxy que loggea automáticamente tokens y coste.

SECCIÓN 7 · RELACIÓN CON OTROS MÓDULOS

- M8-T6 (Inferencia eficiente con vLLM/TGI): la infraestructura de inferencia determina el coste por token en despliegues auto-hosteados.
- M9 (Infraestructura): el análisis cloud vs on-premise se basa en los modelos de coste de inferencia.
- M5-T5 (Tokenización): entender la tokenización es prerequisite para optimizar el uso de tokens.

SECCIÓN 8 · RESUMEN EJECUTIVO

Los tokens de salida cuestan 3-5x más que los de entrada. Las estrategias de optimización de coste más impactantes son: routing por dificultad (15-17x ahorro con mini vs completo en tareas simples), prefix caching del system prompt (ahorro del 30-40% en tokens repetitivos), gestión del historial conversacional (sliding window + resumen, ahorro del 60-70% en sesiones largas), y compresión de prompts (5-10% con

evaluación sobre golden dataset). Monitoriza tokens_input, tokens_output, y coste por consulta con herramientas como Langfuse o Helicone. El break-even cloud vs GPU propio es 6-18 meses a alto volumen.